Lab 5: Ridge and Valley

Overview

We have organized raster analysis around four kinds of operation. A **local** operation computes each output cell from the same cell of the input (unit conversion, a boolean condition, a reclassification). A **focal** operation computes each output cell from a neighborhood of the input (a moving-window mean, a focal range, slope). A **global** operation reduces the whole raster to a number (the minimum, the mean, a percentile) that can then feed a local operation. A **zonal** operation summarizes the raster within each polygon of a vector layer. The week applied all four to `portage_dem.tif`, a DEM of Portage County, Ohio, where the terrain is subdued: 167 m of total relief and a mean slope of about 5 degrees.

This lab applies the same four operations, in the same order, to a landscape with substantial relief. `shenandoah_dem.tif` covers three counties in Virginia's Shenandoah Valley, Page, Shenandoah, and Warren, where the Blue Ridge crest inside Page County exceeds 1,200 m. Two files are used: the DEM, derived from Copernicus global elevation data and reprojected into a projected CRS in meters, and `cbw_counties.geojson`, the 205-county file used since Lab 4.

Download the template notebook, `lab05_template.ipynb`, before you start. It contains one markdown header and one seeded code (or prose) cell per task, in order. Complete your work inside the template and submit the finished file.

Setup

The standard Part 2 installation cell, plus the raster libraries added in week 11:

```
# Run this cell first (takes ~1 minute). Colab already includes geopandas.
%pip install -q libpysal esda splot mapclassify contextily
%pip install -q rasterio rioxarray rasterstats
```

Data. Both files load by URL through the `DATA_BASE_P2` constant:

```
DATA_BASE_P2 = "https://raw.githubusercontent.com/pjbitterman/KSU_x9076/main/data/part2"

# Local path fallback (uncomment if running outside Colab without internet access):
# DATA_BASE_P2 = "../data/part2"
```

- `shenandoah_dem.tif`: a single-band elevation raster in meters, `float32`, with the `-9999` nodata sentinel used by the week 11 DEM. It covers the combined footprint of Page, Shenandoah, and Warren Counties, Virginia. The source resolution is nominally 30 m, but reprojection changes a raster's pixel size; Task A asks for the file's actual pixel size rather than the nominal figure.
- `cbw_counties.geojson`: the 205-county file from Lab 4 and week 9 onward, with `geoid`, `name`, `state`, and geometry. This lab uses three of its rows: Page (`geoid == "51139"`), Shenandoah (`geoid == "51171"`), and Warren (`geoid == "51187"`).

Task A: Open and inspect (10 points)

Open `shenandoah_dem.tif` with rasterio and read its profile before reading any pixel values.

1. Open the file and print its CRS, shape (`rows, cols`), pixel size (`.res`), nodata value, and dtype.
2. Read band 1 into an array, convert nodata cells to `np.nan`, and compute the masked minimum, maximum, and mean elevation with the `np.nan*` functions.
3. In two to three sentences: given the pixel size printed in step 1 (not the nominal 30 m), what can a raster at that resolution show about this terrain, and what can it not? Consider the smallest real feature, such as a streambank or a single ridge outcrop, that could fall entirely within one pixel.

Task B: Clip to one county (10 points)

Task A used the whole file, all three counties at once. This task restricts the raster to Page County with `rasterio.mask.mask`, which clips a raster to a vector geometry (or a list of geometries) and returns a cropped array with its own new transform.

Worked example: clipping a raster to a polygon. The structure of the call, on illustrative inputs:

```
import rasterio
import rasterio.mask

with rasterio.open("some_raster.tif") as src:
    clipped_array, clipped_transform = rasterio.mask.mask(
        src, some_geodataframe.geometry, crop=True
    )
```

Three details matter.

1. `some_geodataframe.geometry` must already be in the raster's CRS: `rasterio.mask.mask` clips by coordinates and does not reproject, so a geometry in geographic degrees applied to a raster in projected meters clips to nothing useful or raises an error.
2. `crop=True` reduces the output array to the geometry's bounding box instead of returning a full-size array with the outside zeroed. `clipped_array` carries a leading band dimension;
3. `clipped_array[0]` is the two-dimensional array to work with, the same shape `.read(1)` returns.

Tasks:

1. Load `cbw_counties.geojson` and select the row where `geoid == "51139"` (Page County).
2. Reproject that one-row GeoDataFrame into the raster's CRS with `.to_crs(...)`.
3. Clip `shenandoah_dem.tif` to the reprojected Page County polygon using the pattern above, and convert the clip's nodata cells to `np.nan`. Every task below works on this clip.
4. Plot the clipped array with `imshow()`, a colorbar labeled in meters, and a title naming the county.

In one sentence: why does this workflow reproject the county polygon to the raster's CRS rather than reprojecting the raster to the county's CRS?

Task C: Focal and global operations (15 points)

Our in-class work built a 3-by-3 moving window from a padded array with `np.lib.stride_tricks.sliding_window_view`, then reduced each window with the `np.nan*` functions. Apply the same pattern to the Page County clip.

1. **Global.** Report the clip's minimum, maximum, mean, and 90th percentile elevation, and the share of valid cells that lie above the mean. State in a comment why that share is not 50%.
2. **Focal mean.** Compute a 3-by-3 focal mean of the clip. Report the mean and maximum absolute difference between the focal mean and the original elevation, and the share of valid cells where that difference exceeds 5 m.
3. **Focal range.** Compute a 3-by-3 focal range (window maximum minus window minimum), a simple roughness measure. Report its mean, maximum, and 95th percentile, and the share of valid cells with a focal range of 30 m or more.
4. Plot the focal range with a colorbar labeled in meters and a title.
5. In two to three sentences: how do the focal-range values for Page County compare with the Portage County values from week 11, and what feature of the landscape do the cells with the largest focal range correspond to?

Handle nodata as the we did in class: pad with `np.nan`, reduce with the `np.nan*` functions, and restore `np.nan` wherever the original cell was nodata.

Task D: Raster algebra, local operations (15 points)

Raster algebra is a local operation - applied to every cell at once. Apply the below functions to the Page County clip and the Task C focal range.

1. **Scalar arithmetic.** Convert the clip to feet and compute relief as height above the clip's minimum. Report the elevation range in feet and the maximum relief in meters.
2. **Reclassification.** Classify elevation into three bands with `np.digitize()`: below 300 m, 300 to 600 m, and 600 m and above. Handle nodata explicitly, as the week 11 session did, and report the share of valid cells in each band.
3. **Boolean combination.** Build a boolean raster of cells that are high **and** rough: elevation of 600 m or more and a focal range (from Task C) of 30 m or more. Report the cell count, the area in km^2 (cell count multiplied by the cell area from Task A's pixel size), and the share of valid cells.
4. Plot the three-class elevation raster from step 2 with a legend naming the classes and a title.
5. In two to three sentences, using the class map from step 4 together with the focal-range map from Task C: where in Page County is the high-and-rough area from step 3, and what part of the Blue Ridge does it correspond to?

Task E: Slope (10 points)

Slope is a focal operation: `numpy.gradient()` computes each cell's rate of change from its neighbors.

1. Compute a slope array in degrees from the clip, following `np.gradient()`, `arctan`, `degrees`, with nodata already converted to `np.nan` before the gradient (a nodata boundary otherwise produces a false cliff-sized slope). Report the minimum, maximum, and mean slope.
2. Report the share of valid cells with a slope of 15 degrees or more.
3. Plot the slope with a colorbar labeled in degrees and a title.
4. In one or two sentences: does the slope map agree with the focal-range map from Task C about where the steep ground is, and which of the two is the better measure of steepness? Give a reason.

Task F: Zonal statistics and a short interpretation (15 points)

Zonal statistics are the fourth category: a summary of the raster within each polygon of a vector layer. In class, we used `rasterstats.zonal_stats()` to join `portage_dem.tif` to municipality polygons. This task runs the same join on all three of this lab's counties, for two rasters: elevation, and a slope array computed on the full `shenandoah_dem.tif` rather than the Page County clip.

1. Select the three rows of `cbw_counties.geojson` where `geoid` is `"51139"` (Page), `"51171"` (Shenandoah), or `"51187"` (Warren), and reproject that subset into the raster's CRS.
2. Compute the full-raster slope array, using the Task E sequence on the whole array from Task A.
3. Run `rasterstats.zonal_stats()` twice against the reprojected three-county subset, once on the full elevation array and once on the full slope array, and extract the mean from each result.
4. Assemble the two means into one DataFrame with one row per county and columns for `geoid`, `name`, mean elevation, and mean slope.

Then, in four to six sentences: which of the three counties is most mountainous by these numbers, and is the answer the same for mean elevation and mean slope? Does it agree with what Tasks C through E showed for Page County? Close with one thing you would check before trusting a 30 m nominal-resolution global DEM for a terrain analysis outside a class exercise.

Deliverables and submission

Submit through Canvas:

1. Your completed template notebook (`lab05_template.ipynb`), containing Task A's inspection output and prose, Task B's clip and plot, Task C's focal and global results, Task D's raster-algebra results, Task E's slope, and Task F's zonal-statistics table and prose, in one file that runs top to bottom.

Submit code as code, not screenshots of code.

Point breakdown

Component	Points
Task A: Open and inspect	10
Task B: Clip to one county	10
Task C: Focal and global operations	15
Task D: Raster algebra, local operations	15
Task E: Slope	10
Task F: Zonal statistics and a short interpretation	15
Total	**75**

Late work follows the course late policy: unless otherwise noted, submissions lose 20% of the possible points per day late.